

simulation evaluation framework

1. Objective

Create a system that can:

- generate synthetic users, posts, and interactions
- simulate honest vs adversarial behavior
- run your full pipeline (Phases 1–5 progressively)
- output quantitative performance metrics

2. Core Architecture

2.1 Components

Simulator Engine → generates events System Engine → runs your algorithms Evaluation Engine → measures performance

2.2 Data Flow

Synthetic Users + Posts ↓ Interaction Generator ↓ Event Stream ↓ Your System (Phases 1–5) ↓ Outputs: credibility, alerts, anomaly ↓ Evaluation Metrics

3. Synthetic Data Model

3.1 User Types

Define distinct behavioral classes:

Honest Users - mostly correct votes - natural timing Noisy Users - random voting Adversarial Users - intentionally wrong - coordinated Bot Users - high rate - low entropy - synchronized

3.2 Parameterization

Each user i has:

$p_{\text{correct}}(i)$ → probability of correct vote $\text{rate}(i)$ → actions per minute $\text{entropy}(i)$ → action diversity
 $\text{coord_group}(i)$ → group membership $\text{location}(i)$ → (optional, Phase 4)

3.3 Post Types

True Posts ($y_j = +1$) False Posts ($y_j = -1$) Ambiguous Posts

Each post has:

difficulty level visibility level initial exposure

4. Interaction Generation

4.1 Time Simulation

for t in time_steps: generate events

4.2 User Selection

sample user i based on activity rate

4.3 Vote Generation

if user is honest: vote = y_j with probability p_{correct} else if adversarial: vote = $-y_j$ else: vote = random

4.4 Coordination Simulation

For bot groups:

for group G: choose post j all users in G vote same value within small time window

4.5 Timing Patterns

Honest → irregular Bots → periodic or bursty

5. Plug Into Your System

At each event:

update_user(i) update_graph(i,j) update_post(j) compute_alerts()

You can toggle phases:

Phase 1 → MVP Phase 2 → +Exp + Anomaly Phase 3 → +Graph Phase 4 → +Location Phase 5 → +ML

6. Evaluation Metrics

6.1 Credibility Accuracy

Accuracy = $\sum_j ((C_j) = y_j)$

6.2 Calibration

Brier Score = $\sum_j (C_j - y_j)^2$

6.3 Early Detection

time_to_correct: time until C_j crosses threshold

6.4 Robustness to Attack

Measure:

attack_success_rate = fraction of false posts classified as true

6.5 User Weight Quality

Corr(w_i , true_reliability_i)

6.6 Anomaly Detection

precision, recall for detecting adversarial users

6.7 Alert Quality

precision = correct alerts / total alerts recall = detected important events / total important events

7. Experiments to Run

Experiment 1: Baseline

Only honest users

Expected:

high accuracy, stable system

Experiment 2: Random Noise

add noisy users

Expected:

minor degradation

Experiment 3: Coordinated Attack

bot group attacks a false post

Compare:

Phase 2 vs Phase 3

Expected:

Phase 3 significantly better

Experiment 4: Location Attack

spoofed location users

Expected:

Phase 4 suppresses influence

Experiment 5: Early Detection

few interactions only

Expected:

Phase 5 improves results

8. Simulation Parameters

num_users num_posts time_horizon attack_fraction group_size interaction_rate_distribution

9. Output Dashboard

Track over time:

$C_j(t)$ $Var_j(t)$ $N_j(t)$ $T_i(t)$ $Anom_i(t)$ alerts triggered

10. Implementation Stack

Option 1 (Fastest)

Python NumPy / Pandas

Option 2 (Scalable)

Python + Kafka (event stream)

11. Minimal Code Skeleton

```
initialize users initialize posts for t in time_steps: events = generate_events(t) for event in events: process_event(event) evaluate_metrics()
```

12. Key Insight

This framework lets you answer:

Does the system actually work under attack?

Not just:

Does the math look correct?

13. Critical Warning

Do NOT skip simulation.

Without it:

- hyperparameters are guesswork
- robustness is unknown
- system may fail in real deployment

14. Final Outcome

After running this, you will know:

- which phase adds real value
- optimal parameter ranges
- system limits